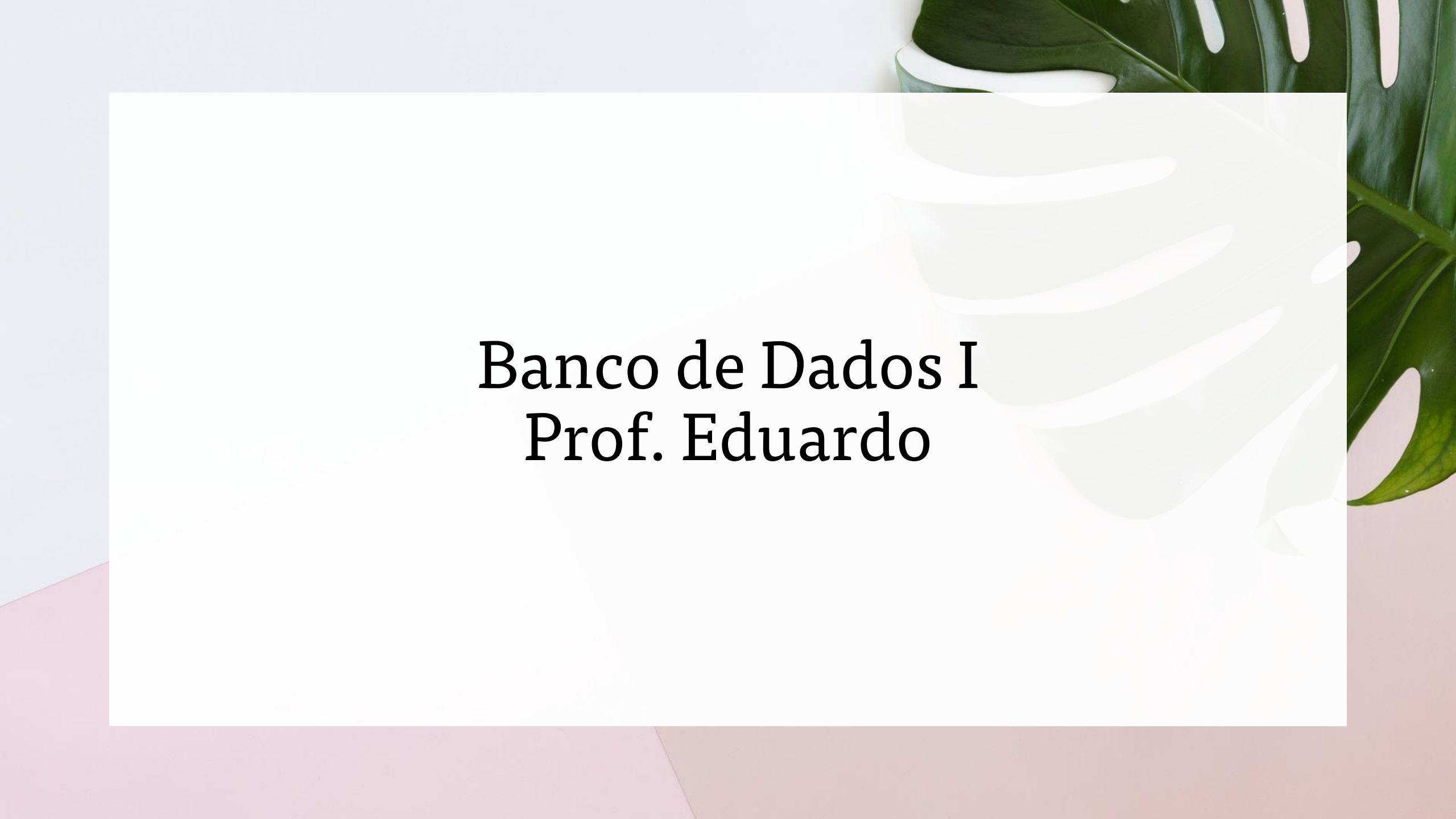




Banco de Dados I

Prof. Eduardo

Conceito

Banco de Dados pode ser entendido como um depósito de um conjunto de registros que são controlados por meio de um computador e que oferece aos usuários vários recursos como: inserção, eliminação e atualização dos dados. Um sistema gerenciador de banco de dados (SGBD) é basicamente o responsável por facilitar o processo de definição e manipulação do banco de dados.

Conceito

Portanto, o SGBD não é o banco de dados, ele é o sistema que gerencia, ou seja, que controla o banco de dados (BD), ele é projetado para fazer o controle de grandes volumes de informações. Em um SGBD podemos ter vários bancos de dados. Nas nossas atividades usaremos o MySQL como SGBD. Porém existem outros no mercado como: Oracle, PostgreSQL, DB2, Informix, SQL Server e outros.

BANCOS DE DADOS RELACIONAIS

Os Bancos de Dados Relacionais foram desenvolvidos para facilitar o acesso aos dados. Pois enquanto em um banco de dados hierárquico os usuários precisam definir as questões de maneira mais específica, iniciando pela raiz, nos Bancos de Dados Relacionais os usuários podem fazer perguntas relacionadas através de vários pontos.

A arquitetura de um banco de dados relacional pode ser descrita usando os termos tabela, linha e coluna. Veja um exemplo;

BANCOS DE DADOS RELACIONAIS

Tabela Clientes	
Código do Cliente	Nome
00001	Antônio Silv
00002	Fábio Cost
00003	Maria Strim

Temos a “Tabela Clientes” que contém duas colunas que são o “Código do Cliente” e o “Nome”, Cada linha é formada por uma lista ordenada de colunas representando um registro.

BANCOS DE DADOS RELACIONAIS

Então um registro é uma instância de uma tabela, ou entidade. Uma entidade é uma representação de um conjunto de informações sobre determinado conceito do sistema. Toda entidade possui atributos, que são as informações que referenciam a entidade.

Se houverem informações a serem armazenadas, você tem uma entidade. Exemplificando: Eu desejo armazenar os seguintes dados do livro: Título, Autor, Editora, Ano, Edição e Volume. Temos então a entidade Livro

BANCOS DE DADOS RELACIONAIS

No exemplo acima “Antônio Silva” é uma instância (registro) da “Tabela Clientes”. As colunas de uma tabela são também chamadas de atributos. Então a coluna “Nome” é um atributo da “Tabela Clientes”.

As tabelas relacionam-se umas as outras através de chaves. Uma chave é um conjunto de um ou mais atributos que determinam a unicidade de cada registro. No nosso caso a chave da “Tabela Clientes” é o “Código do Cliente”, pois ele é único para cada registro.

LINGUAGEM SQL

O nome “SQL” significa “Structured Query Language” que em português quer dizer “Linguagem de Consulta Estrutura”. Essa linguagem teve seus fundamentos no modelo relacional de Codd (1970). Sua primeira versão recebeu o nome de SEQUEL - “Structured English Query Language”, sendo definida principalmente por D.D.CHAMBERLIN, em 1974, nos laboratórios de pesquisa da IBM.

LINGUAGEM SQL

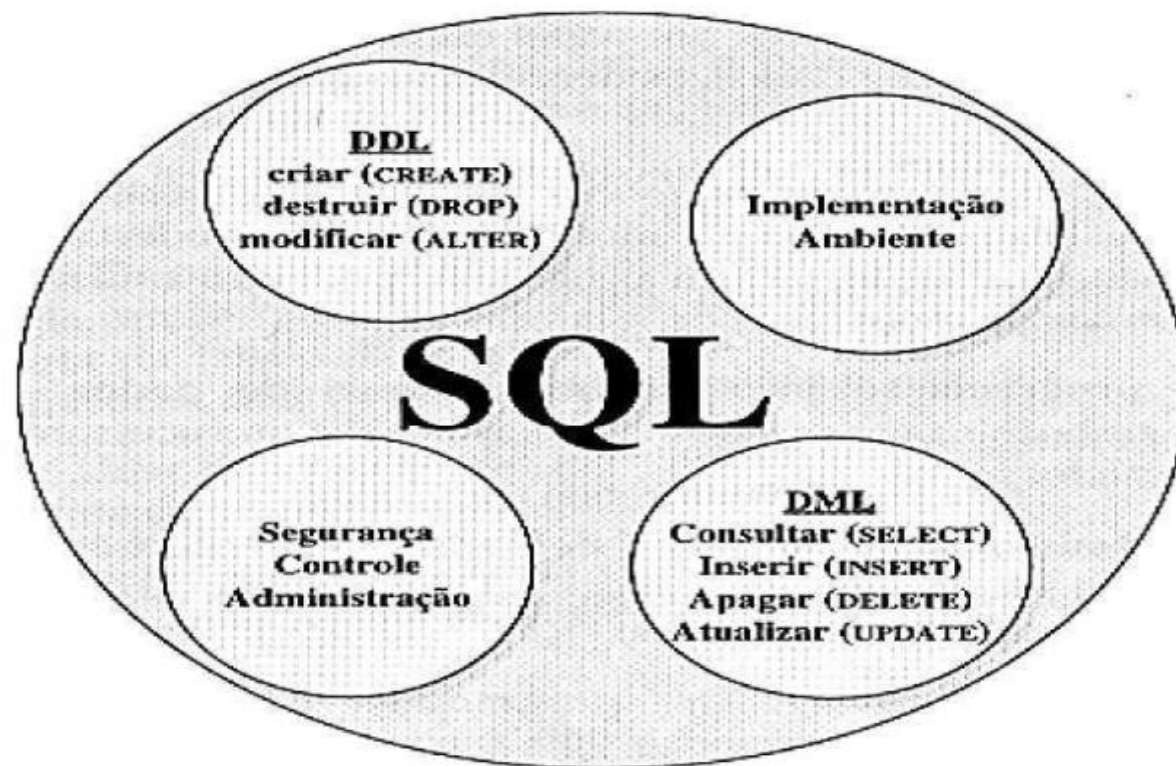
Em 1975, foi testado um protótipo de aplicação dessa nova linguagem. Entre 1976 e 1977, o SEQUEL foi revisado e ampliado, e teve seu nome alterado para “SQL”. Devido ao sucesso dessa nova forma de consulta a manipulação de dados, dentro de um ambiente de banco de dados, a utilização da SQL foi se tornando cada vez maior. Com isso uma grande quantidade de SGBD’s foi tendo como linguagem básica a SQL.

LINGUAGEM SQL

A SQL se tornou um padrão de fato, no mundo dos ambientes de banco de dados relacionais. Em 1982, o American National Standard Institute (ANSI) tornou a SQL padrão oficial de linguagem em ambiente relacional.

Por ser uma linguagem de numerosas aplicações, a SQL pode manipular dados de diferentes bancos entre as funções de um SGBD.

LINGUAGEM SQL



COMPOSIÇÃO DOS BANCOS DE DADOS – DDL e DML

A linguagem SQL é dividida em subconjuntos de acordo com as operações que se deseja efetuar sobre um banco de dados. Os principais subconjuntos são:

COMPOSIÇÃO DOS BANCOS DE DADOS – DDL e DML

DDL - Data Definition Language (Linguagem de Definição de Dados)

O conjunto de comandos da linguagem DDL é usado para a definição das estruturas de dados, fornecendo as instruções que permitem a criação, alteração e remoção de banco de dados, tabelas e etc.

Principais comandos: CREATE, ALTER e DROP

COMPOSIÇÃO DOS BANCOS DE DADOS – DDL e DML

DML - Data Manipulation Language

(Linguagem de Manipulação de Dados)

É o grupo de comandos dentro da linguagem SQL utilizado para a recuperação, inclusão, remoção e modificação de informações em bancos de dados.

Principais comandos: SELECT, INSERT, UPDATE, DELETE, e outros.

COMPOSIÇÃO DOS BANCOS DE DADOS – DDL e DML

COMANDO	DESCRIÇÃO	GRUPO
SELECT	Utilizado para extrair dados do banco	DML
INSERT	Introduzir novas linhas	DML
UPDATE	Alterar linhas já existentes	DML
DELETE	Apagar linhas já existentes	DML
CREATE	Criar estruturas no banco de dados	DDL
ALTER	Alterar estruturas do banco de dados	DDL
DROP	Apagar estruturas do banco de dados	DDL

INTRODUÇÃO AO MySQL

Para conhecer quais os bancos de dados existentes dentro do MySQL basta usar o comando `SHOW DATABASES`. `SHOW` significa mostrar, apresentar, exibir e `DATABASES` significa bases de dados ou bancos de dados, ou seja, manda mostrar os bancos de dados existentes.

Acessando pela primeira vez deve ter pelo menos três que já vem junto da instalação que é o „information_schema“, o „mysql“ e o „test“ que é geralmente fornecido como um espaço de trabalho para usuários fazerem testes.

INTRODUÇÃO AO MySQL

```
MariaDB [(none)]> SHOW DATABASES;
```

Database
carros
concessionaria
escola
information_schema
mysql
performance_schema
phpmyadmin
test

```
8 rows in set (0.001 sec)

MariaDB [(none)]>
```


TIPOS DE DADOS

Strings	
Char	Este tipo de dados armazena uma string de tamanho fixo, com espaços à direita. O tamanho varia de 1 a 255 caracteres. Na consulta, o MySQL ignora os espaços que não foram ocupados.
Varchar	Armazena uma string de tamanho variável, com tamanho mínimo de 1 caracter e máximo de 255.
Tinyblob, Blob, Mediumblob, Longblob	Estes tipos de campo armazenam dados no formato binário.

TIPOS DE DADOS

Muito cuidado, pois o mysql tem como delimitador o ponto e vírgula (;), ou seja, se um ponto e vírgula (;) aparecer em meio a um código, o mysql entende que ali, o comando está encerrado e não há necessidade de continuidade. Muitos erros acontecem por falta do (ponto e vírgula) no final do comando.

No linux, o mysql é case-sensitive por padrão. No Windows não, tanto faz letras maiúsculas como minúsculas. Para desabilitar a sensibilidade do Linux, habilite o parâmetro "lower_case_table_names" no MySQL e pronto.

TIPOS DE DADOS

Data/Hora

Date	Este tipo de dados pode armazenar uma data no formato AAAA-MM-DD
Datetime	Pode armazenar uma data no formato AAAA-MM-DD e uma hora no formato HH:MM:SS
Timestamp	Temos a possibilidade de inserir automaticamente a data/hora atual. Para que isso acontece, basta que ao campo não seja atribuído nenhum valor no momento de inserção do registo.
Time	Armazena um valor horário, no formato HH:MM:SS.
Year	Armazena um numero anual de dois ou quatro dígitos

TIPOS DE DADOS

Numéricos	
Int	Tipo de dados que pode variar de -2147483648 a 2147483647 e de 0 a 4294967295, caso o parâmetro UNSIGNED seja utilizado.
Float	Armazena um número do tipo ponto flutuante de precisão simples. Varia de -3.402823466E+38 a -1.175494351E-38, 0, e 1.175494351E-38 a 3.402823466E+38
Double	Armazena um número do tipo ponto flutuante de precisão dupla. Varia de -1.7976931348623157E+308 a -2.2250738585072014E-308, 0, e 2.2250738585072014E-308 a 1.7976931348623157E+308.
Decimal	Tipo de dados que se comporta como o do tipo CHAR, ou seja, cada dígito ocupa 1 byte.

INSTRUÇÕES DDL – CREATE

Instrução que permite a criação de um banco de dados e também as demais estruturas de um banco. As duas situações onde o CREATE é utilizado com maior frequência são:

1. Criação de um novo banco de dados
2. Criação de uma nova tabela

```
mysql> CREATE DATABASE escola;  
Query OK, 1 row affected (0.00 sec)
```

1º Passo: Abra o MySQL e digite o comando CREATE DATABASE <nome do banco>;

INSTRUÇÕES DDL – CREATE

2º Passo: Para que você possa ter acesso a qualquer informação do banco de dados, ou mesmo criar tabelas, inserir registro ou fazer uma consulta você precisa entrar dentro dele. O comando para isso é USE <nome do banco>;



```
mysql> USE escola;  
Database changed
```


INSTRUÇÕES DDL – CREATE

3º Passo: Agora como você já tem o banco de dados e entrou nele, você já pode começar a criar as tabelas do seu banco. Para criar tabelas é preciso seguir um padrão básico:

CREATE TABLE <nome da tabela>(
<nome da coluna> <tipo de dados> (<quantidade de caracteres>),
<nome da coluna> <tipo de dados> (<quantidade de caracteres>) ;

```
mysql> CREATE TABLE aluno(  
    -> nome VARCHAR(50),  
    -> idade INT(2));  
Query OK, 0 rows affected (0.04 sec)
```


INSTRUÇÕES DDL – CREATE

Esse é um exemplo simples da criação de uma tabela do BD, você pode de acordo com a situação acrescentar mais campos e defini-los com mais características. Por exemplo, a coluna nome pode ser definida como NOT NULL, ou seja, não pode ser nulo(vazio).

Quando for inserido um novo aluno o nome dele vai ser obrigatório. Pense comigo, na matrícula de um novo aluno o seu nome não pode faltar e o numero da sua matrícula é gerada automaticamente pelo sistema.

INSTRUÇÕES DDL – CREATE

Para a criação do número da matrícula automático é usado o código `AUTO_INCREMENT`, esse código permite que o número da matrícula seja acrescentado a cada novo registro inserido na tabela, ou seja, a cada novo aluno matriculado.

Como a matrícula é um número que não se repete e que é único para cada aluno ele pode ser declarado como chave primária. Em banco de dados relacionais, a chave primária é muito importante para quando for fazer o relacionamento entre as tabelas. Mais na frente iremos falar mais detalhado sobre chaves.

INSTRUÇÕES DDL – CREATE

Ao ser definida uma coluna como chave primária, estamos dizendo para o banco de dados que não podem existir duas linhas nessa coluna com um mesmo valor. Geralmente quando falamos de chave primária usamos a sigla PK referente à palavra PRIMARY KEY.

```
mysql> CREATE TABLE novo_aluno(  
-> matricula INT(10) AUTO_INCREMENT,  
-> aluno VARCHAR(50) NOT NULL,  
-> idade INT(2),  
-> PRIMARY KEY (matricula));  
Query OK, 0 rows affected (0.06 sec)
```


INSTRUÇÕES DDL – CREATE

4º Passo:

Para ver como ficou a estrutura das tabelas criadas usamos o código `DESCRIBE TABLE <nome da tabela>;` ou somente `DESC TABLE <nome da tabela>;` Mais se você quer ver como foi criada a tabela pode usar o comando `SHOW CREATE TABLE <nome da tabela>`

INSTRUÇÕES DDL – CREATE

4º Passo:

Exemplo DESCRIBE:

```
mysql> DESCRIBE novo_aluno;
```

Field	Type	Null	Key	Default	Extra
matricula	int(10)	NO	PRI	NULL	auto_increment
aluno	varchar(50)	NO		NULL	
idade	int(2)	YES		NULL	

```
3 rows in set (0.01 sec)
```


INSTRUÇÕES DDL – CREATE

4º Passo:

Exemplo DESC:

```
mysql> DESC novo_aluno;
```

Field	Type	Null	Key	Default	Extra
matricula	int(10)	NO	PRI	NULL	auto_increment
aluno	varchar(50)	NO		NULL	
idade	int(2)	YES		NULL	

```
3 rows in set (0.01 sec)
```


INSTRUÇÕES DDL – CREATE

4º Passo: Exemplo SHOW CREATE:

```
mysql> SHOW CREATE TABLE novo_aluno;
```

Table	Create Table
novo_aluno	CREATE TABLE `novo_aluno` (`matricula` int(10) NOT NULL AUTO_INCREMENT, `aluno` varchar(50) NOT NULL, `idade` int(2) DEFAULT NULL, PRIMARY KEY (`matricula`)) ENGINE=InnoDB DEFAULT CHARSET=latin1

1 row in set (0.00 sec)


```

mysql> CREATE DATABASE escola;
Query OK, 1 row affected (0.00 sec)

mysql> USE escola;
Database changed
mysql> CREATE TABLE aluno(
  -> nome VARCHAR(50),
  -> idade INT(2));
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE novo_aluno(
  -> matricula INT(10) AUTO_INCREMENT,
  -> aluno VARCHAR(50) NOT NULL,
  -> idade INT(2),
  -> PRIMARY KEY(matricula));
Query OK, 0 rows affected (0.01 sec)

mysql> DESCRIBE novo_aluno;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| matricula | int(10) | NO | PRI | NULL | auto_increment |
| aluno | varchar(50) | NO | | NULL | |
| idade | int(2) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)

mysql> DESCRIBE aluno;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| nome | varchar(50) | YES | | NULL | |
| idade | int(2) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)

mysql>

```

Veja agora como ficou o código criado e compare com o que você fez.

INSTRUÇÕES DDL – CREATE

Exercícios:

Com base no exemplo anterior crie um novo banco de dados chamado biblioteca.

- Criar a tabela livros contendo os seguintes campos:
- Código: inteiro com no máximo 8 dígitos
- Título: máximo de 40 caracteres
- Autor: máximo de 50 caracteres
- Editora: máximo de 20 caracteres
- Edição: inteiro com no máximo 2 dígitos
 - ✓ O código é a chave primária e deve ser gerado automaticamente
 - ✓ O Título e o Autor não podem ser vazios.

INSTRUÇÕES DDL – ALTER

Já aprendemos a criar tabelas no BD. Mas, como proceder para inserir, por exemplo, uma coluna em uma tabela já existente. Será que é necessário para isso excluir toda a tabela? A resposta para essa pergunta é não. Utilizando o comando ALTER, é possível que o usuário altere a estrutura de uma tabela, por exemplo, inserindo novas colunas.

Usando o comando ALTER, é possível realizar as seguintes alterações na estrutura de uma tabela:

1. Adicionar colunas;
2. Excluir colunas;
3. Alterar o tipo e o nome de uma coluna já existente.

INSTRUÇÕES DDL – ALTER

Utilizando o banco de dados escola e a tabela novo_aluno vamos adicionar uma coluna chamada de endereco. Para adicionar colunas na tabela seguimos um padrão básico:

```
ALTER TABLE <nome da tabela> ADD <nome da coluna>  
<tipo de dado>;
```

OBS: No comando ALTER aparece a cláusula ADD indicando que estamos adicionando uma coluna à tabela e definindo o seu tipo.

INSTRUÇÕES DDL – ALTER

Exemplo ADD:

```
mysql> use escola;  
Database changed  
mysql> ALTER TABLE novo_aluno ADD endereco VARCHAR(150);  
Query OK, 0 rows affected (0.63 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> DESCRIBE novo_aluno;
```

Field	Type	Null	Key	Default	Extra
matricula	int(10)	NO	PRI	NULL	auto_increment
aluno	varchar(50)	NO		NULL	
idade	int(2)	YES		NULL	
endereco	varchar(150)	YES		NULL	

```
4 rows in set (0.05 sec)
```


INSTRUÇÕES DDL – ALTER

Perceba que a coluna endereço foi adicionada na ultima posição, porém nem sempre queremos que a coluna que adicionarmos fique por ultimo, por isso Você pode utilizar palavras chaves como FIRST (primeiro) ou AFTER (após) para posicionar a nova coluna na posição que desejar na tabela.

INSTRUÇÕES DDL – ALTER

Exemplo FIRST:

Usando o mesmo banco de dados, porém na tabela aluno vamos adicionar a coluna código como sendo a primeira coluna e chave primaria da tabela.

```
mysql> ALTER TABLE aluno ADD codigo INT(8) AUTO_INCREMENT PRIMARY KEY FIRST;  
Query OK, 0 rows affected (0.11 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> DESCRIBE aluno;
```

Field	Type	Null	Key	Default	Extra
codigo	int(8)	NO	PRI	NULL	auto_increment
nome	varchar(50)	YES		NULL	
idade	int(2)	YES		NULL	

```
3 rows in set (0.01 sec)
```


INSTRUÇÕES DDL – ALTER

Exemplo AFTER:

O FIRST insere na primeira coluna, já com o AFTER você escolhe a posição que deseja que a coluna fique, ou melhor, depois de quem a coluna vai ficar. Usando a mesma tabela aluno vamos inserir a coluna nascimento depois da coluna nome.

```
mysql> ALTER TABLE aluno ADD nascimento DATE AFTER nome;  
Query OK, 1 row affected (0.07 sec)  
Records: 1 Duplicates: 0 Warnings: 0
```

```
mysql> DESCRIBE aluno;
```

Field	Type	Null	Key	Default	Extra
codigo	int(8)	NO	PRI	NULL	auto_increment
nome	varchar(50)	YES		NULL	
nascimento	date	YES		NULL	
idade	int(2)	YES		NULL	

```
4 rows in set (0.01 sec)
```


INSTRUÇÕES DDL – ALTER

Agora suponhamos que a coluna idade da tabela aluno seja desnecessária, então, neste caso, será possível também apagar a coluna da tabela. Para excluir uma coluna usamos o comando `ALTER TABLE <nome da tabela> DROP <nome da coluna>;` Perceba que agora usamos o `DROP`, em vez de `ADD`.

INSTRUÇÕES DDL – ALTER

Exemplo DROP:

```
mysql> ALTER TABLE aluno DROP idade;  
Query OK, 1 row affected (0.02 sec)  
Records: 1 Duplicates: 0 Warnings: 0
```

```
mysql> SHOW FIELDS FROM aluno;
```

Field	Type	Null	Key	Default	Extra
codigo	int(8)	NO	PRI	NULL	auto_increment
nome	varchar(50)	YES		NULL	
nascimento	date	YES		NULL	

3 rows in set (0.01 sec)

OBS: o comando SHOW FIELDS tem a mesma função do DESCRIBE.